A1-2. Field Verification System Design Document [Receiver]

October 10, 2026 — Prepared by Attractor Co., Ltd.

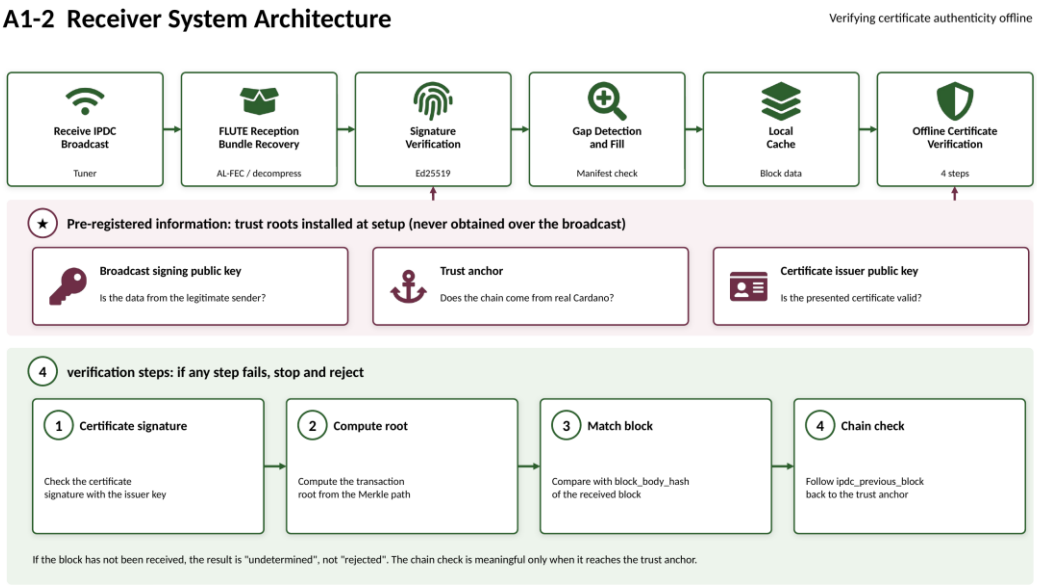
> This document is the Milestone 1 deliverable "A1-2. Field Verification System Design Document [Receiver]" and satisfies acceptance criterion A2 (it includes a system diagram, hardware definitions, and software function descriptions).

Overview and Positioning

This document corresponds to the Milestone 1 deliverable "A1-2. Field Verification System Design Document [Receiver]" and defines the receiver-side system (from IPDC reception by multiple receivers to offline certificate verification).

The scope covers reception of IPDC broadcast waves, local storage of the received block information, offline matching against a presented certificate, and display of the verification result. The sender-side design is defined in A1-1.

System Diagram



Receiver system architecture

The receiver-side roles are the blocks shown in the diagram above. The pre-registered information is stored in the receiver at installation and is never obtained over the broadcast; it serves as the root of trust for verification. When multiple receivers are used, the configuration from the tuner onward is duplicated for each receiver (corresponding to Tests 2 and 3 in B1), and the verification results of each receiver are recorded separately.

Hardware Definitions

Component	Expected Equipment / Specifications	Role
IPDC tuner / receiver module	Tuner for digital terrestrial broadcast reception (data broadcasting / IPDC capable). Assumes a configuration equivalent to the indoor receiver in the MIC "Technical Guidelines for Disaster	Receives broadcast waves and demodulates them into a baseband signal

Component	Expected Equipment / Specifications	Role
	Information Transmission Using Digital Terrestrial Broadcast Waves" (model to be confirmed)	
Decoder board / embedded device	Embedded Linux device or general-purpose PC with a receiver board (to be confirmed)	Extracts and decodes block information from the demodulated signal
Local storage	Built-in storage of the embedded device, an SD card, etc.	Temporarily caches the received block information
Certificate presentation interface	For the experiment, the certificate JSON file is loaded into the verification terminal (the same method as the previous project). Presentation by QR code or IC card will be considered for future production use	Inputs the certificate to be verified into the system
Verification terminal (application device)	Tablet or embedded terminal. For the experiment, the same model is used at all three sites (model to be confirmed)	Runs the offline certificate verification function and displays the result
Pre-registered information storage	For the experiment, stored as a read-only file in the receiver. Protection with a secure element is a requirement for future production use	Holds the broadcast signature verification public key, the known block header serving as the trust anchor, and the certificate issuer's public key in a tamper-proof form
Battery backup	Not required for the experiment (field tests run on mains power or a standard portable power supply). Continuous operation for 24 hours or more is a requirement for future production use	Keeps reception and verification running during a power outage (in line with the indoor receiver requirements of the MIC guideline)
Multiple receiver sets	Three sets of the above (one at each of the three sites)	For simultaneous reception at three sites in the field test

Software Function Descriptions

Function	Description	Input	Output	Development
IPDC reception and demodulation	Receives broadcast waves and demodulates them into a baseband signal	IPDC broadcast waves	Demodulated signal	Done (existing implementation)
FLUTE reception and bundle recovery	Recovers received FLUTE objects using AL-FEC, decompresses them, and splits them into individual block header JSON documents and manifests	Demodulated signal	Block header JSON documents and manifests	Done (existing implementation)
Signature verification	Verifies the Ed25519 signatures of each bundle's manifest and each block header JSON with the pre-registered broadcast signature verification public key. Data that fails verification is not accepted, and the reason for rejection is recorded	Block header JSON documents, manifests, and the pre-registered public key	Signature-verified block headers	New development
Gap detection and filling	Detects gaps by comparing the expected heights in the manifest with the blocks held, and fills them from subsequent bundles received through overlapping transmission. Gaps that cannot be filled are recorded as "not received," distinguished from tampering	Signature-verified block headers and manifests	Completed block information and gap list	Update (Modification of a previous project)
Local cache	Stores signature-verified block information locally for a set period. When a block of the same height is received more	Block header data	Cached block information	Update (Modification of a previous project)

Function	Description	Input	Output	Development
	than once, checks that the contents match, and records a mismatch as a rollback or tampering candidate			
Certificate reading	Reads the presented certificate (JSON containing the transaction information and Merkle path)	Certificate JSON file	Certificate data	Update (Modification of a previous project)
Offline certificate verification	Verifies authenticity in four steps: verifying the certificate signature, computing the transaction root from the Merkle path, matching it against block_body_hash of the received block, and verifying the chain back to the trust anchor (details in the next section)	Certificate data, cached block information, and pre-registered information	Verification result (authentic / rejected / undetermined)	Update (Modification of a previous project)

Offline Verification Algorithm

Pre-registered Information

The receiver holds the following three items from installation. They serve as the roots of trust for verification, and none of them is obtained over the broadcast.

Pre-registered Information	Purpose
Broadcast signature verification public key (Ed25519)	Confirms that received data was sent by the legitimate source
Trust anchor (height and ipdc_hash of a known, trusted block)	Confirms that the received chain originates from the real Cardano chain
Certificate issuer's public key	Verifies the signature of the presented certificate itself

Verification Procedure

When a certificate is presented, it is verified in the following four steps. If any step fails, the remaining steps are not executed, the failed step is recorded, and the certificate is rejected.

1. **Certificate signature verification** — Verify the signature of the presented certificate with the certificate issuer's public key.
2. **Transaction root computation** — Compute the leaf hash of the transaction contained in the certificate and combine it in order with the enclosed Merkle path to compute the root value. The hash function and combination rules must be identical to those on the sender side (SHA3-256; leaves are transaction IDs as hexadecimal strings; internal nodes hash the concatenation of the left and right hexadecimal strings).
3. **Matching against the block header** — Retrieve the block header at the block height referenced by the certificate from the local cache and compare its block_body_hash with the computed root value. If the block has not been received, the result is distinguished as "undetermined," not rejected.
4. **Chain verification back to the trust anchor** — Follow ipdc_previous_block from that block, recomputing ipdc_hash for each block to confirm that the links hold, and confirm that the pre-registered trust anchor is reached.

Why Chain Verification Alone Is Not Enough

Because ipdc_hash can be computed from public information alone, a third party can generate a self-consistent sequence of fake block headers

from arbitrary initial values. In fact, when a fake chain built from random values unrelated to Cardano was fed into logic identical to the previous project's verification tool, every block was judged as successfully verified.

Chain verification in step 4 is therefore meaningful only when combined with confirmation that the pre-registered trust anchor is reached. In addition, none of steps 1–4 can confirm that the received data was sent by the legitimate source, so signature verification at the time of reception is required separately.

Summary

This document defined the receiver-side system that receives block information from broadcast waves and verifies the authenticity of certificates without connecting to the Internet. The receiver consists of an IPDC tuner, a decoder device, local storage, and a verification terminal, and the same configuration is installed at each of the three sites in the field test. The experiment uses a minimal configuration: certificates are presented as JSON files, and the pre-registered information is held in a read-only file in the receiver.

Received FLUTE objects are recovered bundle by bundle, and their signatures are verified before storage. Gaps are detected by comparison with the manifest and filled from adjacent bundles received through overlapping transmission. Gaps that cannot be filled are distinguished from tampering and treated as "not received."

For certificate verification, three pre-registered items serve as the roots of trust: the broadcast signature verification public key, the trust anchor, and

the certificate issuer's public key. Certificates are then judged in four steps: certificate signature verification, root computation, matching against the block, and chain verification back to the trust anchor. Tests with the previous project's verification tool confirmed that chain consistency alone cannot detect a fake chain, so signature verification and trust anchor matching are mandatory.

Remaining items, such as the models of the receiver and verification terminal, the installation locations of the three sites, the registration and update procedures for pre-registered information, and how revocation information is applied, will be finalized in the M2 detailed design.